Docelowy model

Cel:

$$\max \; Revenue_{DA} + Revenue_{ID} + Revenue_{FCR} + Revenue_{aFRR} + Revenue_{mFRR} - Cost_{degradation}$$

Model:

Polska, BESS standalone, bez forecastowania cen, z historycznymi danymi rynkowymi.

Docelowo:

Element	Założenie
fizyczny timestep	**15 min**
optimization horizon	**24 h**
rolling step	**1 h**
DA	tak
Intraday / RDB	tak
FCR	tak, późniejszy etap
aFRR UP/DOWN	tak
mFRR UP/DOWN	tak
reservation	wpływa na dostępne MW i SoC headroom
activation	wpływa na rzeczywisty SoC
forecast cen	**nie**
solver	MILP/LP, np. HiGHS

1. Najpierw dokładnie zdefiniuj zasady rynku

Zanim zaczniesz kodować optimizer, zrób jedną tabelę `market_rules`.

Dla każdego produktu ustal:

Produkt	direction	jednostka ceny	długość produktu	gate closure	minimalny bid	activation
DA	buy/sell	PLN/MWh	15 min	...	...	—
ID	buy/sell	PLN/MWh	15 min	...	...	—
FCR	UP/ DOWN	PLN/MW/h	wg zasad PSE	...	...	częstotliwość

Produkt	direction	jednostka ceny	długość produktu	gate closure	minimalny bid	activation
aFRR	UP	PLN/MW/h + PLN/MWh	wg zasad PSE	…	…	automatyczna
aFRR	DOWN	PLN/MW/h + PLN/MWh	…	…	…	automatyczna
mFRR	UP	PLN/MW/h + PLN/MWh	…	…	…	manual/direct
mFRR	DOWN	PLN/MW/h + PLN/MWh	…	…	…	manual/direct

Nie kopiowałbym automatycznie sześciu bloków 4-godzinnych z modelu niemieckiego. Długość produktów i okresów nabywania trzeba ustawić zgodnie z polskimi WDB.

Aktualne WDB PSE rozdzielają FCR oraz FRR, w tym aFRR i mFRRd, a moce bilansujące są nabywane oddzielnie dla kierunku w górę i w dół.

---

2. Ustal okres backtestu

Najczystszy dataset zacząłbym od:

$$\boxed{1\ \text{października}\ 2025}$$

Powód: od tego dnia SDAC działa z **15-minutowym MTU**, więc DA, fizyka baterii i balancing można trzymać na wspólnej osi 15-minutowej.

To jest prostsze niż mieszanie:

- godzinowego DA,
- 15-minutowego balancing,
- 15-minutowego ID.

Najpierw zrobiłbym np. **3 miesiące** jako development dataset.

Dopiero kiedy wszystko działa:

pełny rok.

---

3. Zbuduj jeden master dataset 15-minutowy

Docelowo chcesz mieć coś w tym stylu:

```
datetime
da_price
```

```
id_price

fcr_capacity_price_up
fcr_capacity_price_down

afrr_capacity_price_up
afrr_capacity_price_down
mfrr_capacity_price_up
mfrr_capacity_price_down

afrr_energy_price_up
afrr_energy_price_down
mfrr_energy_price_up
mfrr_energy_price_down

afrr_activation_up
afrr_activation_down
mfrr_activation_up
mfrr_activation_down
```

Plus ewentualnie:

```
fcr_procured_up
fcr_procured_down

afrr_procured_up
afrr_procured_down

mfrr_procured_up
mfrr_procured_down
```

jeżeli potrzebujesz ich do oszacowania activation ratio.

---

4. Pobierz DA

Najpierw zbuduj najłatwiejszy strumień danych:

$$price_t^{DA}$$

PSE ma obecnie API v2 i m.in. endpoint dotyczący ceny SDAC (`csdac-pln`). W tym samym API znajduje się już większość danych, których później będziesz potrzebował do rynku bilansującego.

DA powinno finalnie wyglądać:

```
datetime            da_price
2025-10-01 00:00    ...
2025-10-01 00:15    ...
```

```
2025-10-01 00:30     ...
2025-10-01 00:45     ...
```

---

5. Pobierz capacity prices dla FCR/aFRR/mFRR

Tutaj PSE jest bardzo wygodne.

W API masz między innymi:

```
cmbp-tp
```

czyli ceny mocy bilansujących nabytych w trybie podstawowym.

Endpoint zawiera osobno m.in.:

```
fcr_g
fcr_d

afrr_g
afrr_d

mfrrd_g
mfrrd_d
```

oraz odpowiadające im timestampy.

Analogicznie masz:

```
mbp-tp
```

dla wolumenów mocy oraz:

```
cmbu-tu
mbu-tu
```

dla trybu uzupełniającego. Katalog PSE API potwierdza dostępność tych endpointów.

Na początku możesz uprościć model do **MBP**, a MBU dodać później.

---

6. Pobierz dane o activation i balancing energy

Tu potrzebujesz dwóch rzeczy osobno:

A. Ile zostało aktywowane

np.:

$$E_{system,t}^{aFRR,UP}$$

B. Po jakiej cenie

$$P_{energy,t}^{aFRR,UP}$$

W API PSE dostępne są m.in. endpointy związane z energią bilansującą i jej cenami (`eb-rozl`, `energy-prices` oraz pozostałe raporty RB).

Same WDB pokazują też wprost, że energia bilansująca wynikająca z aktywacji aFRR jest liczona z aktywowanej mocy w poszczególnych okresach aktywacji.

Na tym etapie najpierw zbuduj **historyczny profil systemowej aktywacji**.

Przy uproszczeniu:

$$\alpha_t^{aFRR,UP} = \frac{E_{system,t}^{aFRR,UP}}{R_{system,t}^{aFRR,UP}\Delta t}$$

Potem:

$$E_{BESS,t}^{aFRR,UP} = R_{BESS,t}^{aFRR,UP}\alpha_t^{aFRR,UP}\Delta t$$

Analogicznie dla DOWN i mFRR.

Ważne: to jest przybliżenie proporcjonalnej aktywacji. W realnym rynku aktywacja zależy od ofert energii i merit orderu. Dlatego `alpha` traktujesz jako scenariusz historycznej aktywacji systemu, nie dokładną rekonstrukcję pracy konkretnej baterii.

To wystarczy jako sensowna pierwsza wersja pracy.

---

7. Zdobądź Intraday / RDB

To będzie prawdopodobnie najtrudniejszy dataset.

TGE AIR oferuje historyczny produkt dotyczący wyników Rynku Dnia Bieżącego; dokumentacja AIR pokazuje wprost raport **„Wyniki sesji Rynku Dnia Bieżącego dla energii elektrycznej”**. Dostęp do takich raportów może być płatny.

Są dwie wersje, które możesz zrobić.

Wersja prosta:

```
delivery_time
RDB_VWAP
```

czyli jedna reprezentatywna cena ID dla każdego MTU.

To wystarcza do:

perfect-foresight co-optimization DA + ID + balancing.

Wersja docelowa dla prawdziwego rollingu:

```
transaction_time
delivery_time
price
volume
```

albo snapshoty order booka.

Wtedy wiesz np.:

```
16:00 -> cena produktu 18:00 = 420
16:15 -> 425
16:30 -> 411
16:45 -> 437
```

i optimizer o 16:15 nie ma dostępu do ceny z 16:45.

Jeżeli nie zdobędziesz timestampowanych danych ID, nadal możesz zrobić rolling, ale będzie to **rolling perfect foresight**, a nie realistyczny backtest tradingowy.

---

8. Zbuduj najpierw sam model fizyczny baterii

Bez żadnego rynku.

Parametry:

```
P_max
E_max

soc_min
soc_max
soc_initial

eta_charge
eta_discharge
```

```
degradation_cost
```

Zmienne:

```
charge[t]
discharge[t]
soc[t]
```

Podstawowy constraint:

$$SOC_{t+1} = SOC_t + \eta_c P_t^{ch} \Delta t - \frac{P_t^{dis} \Delta t}{\eta_d}$$

oraz:

$$SOC_{min} \leq SOC_t \leq SOC_{max}$$

$$0 \leq P_t^{ch} \leq P_{max}$$

$$0 \leq P_t^{dis} \leq P_{max}$$

Jeżeli potrzebujesz, dodajesz binary zapobiegające jednoczesnemu charge/discharge.

Bardzo ważny będzie też **terminal SoC constraint** albo terminal value, żeby optimizer nie opróżniał baterii pod koniec każdego horyzontu.

---

9. Dodaj tylko Day-Ahead

Teraz:

```
da_buy[t]
da_sell[t]
```

i:

$$Revenue_{DA} = \sum_t P_t^{DA} (E_{DA,t}^{sell} - E_{DA,t}^{buy})$$

To jest Twój pierwszy działający optimizer.

Na nim sprawdzasz:

- czy SoC działa,
- czy efficiency działa,
- czy bateria ładuje się tanio i rozładowuje drogo,
- czy nie przekracza mocy,
- czy poprawnie liczysz revenue.

Nie przechodź dalej, dopóki ten model nie jest w 100% zwalidowany.

---

10. Dodaj reservation FCR/aFRR/mFRR

Nowe zmienne:

```
fcr_up[t]
fcr_down[t]

afrr_up[t]
afrr_down[t]

mfrr_up[t]
mfrr_down[t]
```

Capacity revenue:

$$Revenue_{aFRR}^{cap} = \sum_t R_t^{aFRR} C_t^{aFRR} \Delta t$$

Analogicznie dla pozostałych.

Teraz kluczowe constrainty mocy:

$$P_t^{discharge} + R_t^{FCR,UP} + R_t^{aFRR,UP} + R_t^{mFRR,UP} \leq P_{max}$$

oraz:

$$P_t^{charge} + R_t^{FCR,DOWN} + R_t^{aFRR,DOWN} + R_t^{mFRR,DOWN} \leq P_{max}$$

Czyli reservation **nie zmienia SoC**, ale blokuje moc.

Do tego energy headroom:

$$SOC_t \geq SOC_{min} + R_t^{UP} T_{UP}^{req}$$

oraz:

$$SOC_t \leq SOC_{max} - R_t^{DOWN} T_{DOWN}^{req}$$

z poprawką na efficiency.

Czyli reservation blokuje również odpowiedni **bufor energii**.

Na tym etapie dodajesz też constrainty długości produktu:

jeżeli oferta aFRR musi być stała w określonym bloku, to:

$$R_t = R_{t+1} = ... = R_{t+n}$$

zgodnie z **polskimi** zasadami produktu.

---

11. Dopiero teraz dodaj activation

Activation NIE jest zmienną decyzyjną.

Optimizer wybiera:

$$R_t^{aFRR,UP}$$

a historia mówi mu:

$$\alpha_t^{aFRR,UP}$$

więc:

$$E_t^{aFRR,UP} = R_t^{aFRR,UP}\alpha_t^{aFRR,UP}\Delta t$$

Activation revenue:

$$Revenue_{aFRR,UP,t}^{act} = E_t^{aFRR,UP}P_{energy,t}^{aFRR,UP}$$

i najważniejsze:

$$SOC_{t+1} = SOC_t - \frac{E_t^{aFRR,UP}}{\eta_d}$$

Dla DOWN:

$$SOC_{t+1} = SOC_t + \eta_c E_t^{aFRR,DOWN}$$

Tak samo mFRR.

Otrzymujesz więc rozdzielenie:

```
RESERVATION
    ↓
capacity revenue
    ↓
blokuje MW + SoC headroom

ACTIVATION
    ↓
energy revenue
    ↓
faktycznie zmienia SoC
```

---

12. Dodaj Intraday do jednego globalnego solve

Zanim zrobisz rolling, najpierw zrób **perfect foresight**.

Dodajesz:

```
id_buy[t]
id_sell[t]
```

i:

$$P_t^{baseline} = P_t^{DA} + P_t^{ID}$$

Fizyczna praca baterii wynika więc z DA + ID + activation.

Funkcja celu:

$$\max Revenue_{DA} + Revenue_{ID} + Revenue_{capacity} + Revenue_{activation} - Cost_{degradation}$$

Najpierw rozwiąż np. cały tydzień z pełną znajomością cen.

To jest świetny benchmark:

$$\boxed{Maximum\ achievable\ revenue}$$

---

13. Porównaj cztery podstawowe strategie

Zanim wejdziesz w rolling, policz:

Model	Markets
M1	DA
M2	DA + ID
M3	DA + FCR + aFRR + mFRR
M4	DA + ID + FCR + aFRR + mFRR

I policz:

$$Value_{ID} = Revenue_{M4} - Revenue_{M3}$$

To może być jeden z głównych wyników pracy.

---

14. Dopiero wtedy zbuduj rolling optimizer

Base case:

$$\boxed{\Delta t = 15min, \quad rolling = 1h, \quad horizon = 24h}$$

Czyli:

```
10:00
solve 10:00 → jutro 10:00

11:00
solve 11:00 → jutro 11:00

12:00
solve 12:00 → jutro 12:00
```

Każdy solve ma:

$$24h \times 4 = 96$$

15-minutowych okresów.

W każdym rollingu:

A. bierzesz aktualny rzeczywisty `SOC`;

B. wcześniejsze transakcje DA są fixed;

C. zawarte już transakcje ID są fixed;

D. zobowiązania FCR/aFRR/mFRR, których gate closure minęło, są fixed;

E. okresy ID, których gate closure minęło, są fixed;

F. pozostałe zmienne optimizer może zmieniać;

G. rozwiązujesz następne 24 h;

H. wykonujesz tylko najbliższą godzinę;

I. podczas tej godziny co 15 min realizujesz historyczną activation i aktualizujesz faktyczny SoC;

J. po godzinie zaczynasz nowy solve od:

$$SOC_0 = SOC^{actual}$$

Schemat:

```
OPTIMIZE 24H
     ↓
execute 1h
```

```
     ↓
actual aFRR/mFRR activation
     ↓
actual SOC
     ↓
+1 hour
     ↓
OPTIMIZE 24H AGAIN
```

---

15. Bardzo jasno rozdziel dwa rodzaje rollingu

Jeżeli nie robisz forecastów, masz dwie możliwości.

Rolling Perfect Foresight

Optimizer w każdym 24-h horyzoncie zna przyszłe historyczne ceny.

To nie jest realistyczna strategia tradingowa.

To jest:

$$\boxed{\text{upper bound}}$$

i jest całkowicie OK jako analiza akademicka.

Realistic Sequential Backtest

Optimizer w chwili t zna tylko informacje dostępne w chwili t.

Do tego potrzebujesz przede wszystkim timestampowanych danych RDB/order book.

Bez nich nie udawaj, że model jest realistycznym trading algorithmmem.

Najlepiej zrobić Perfect Foresight jako główny model i realistic rolling tylko wtedy, gdy dane ID na to pozwolą.

---

16. FCR dodałbym dopiero na końcu

aFRR/mFRR są łatwiejsze koncepcyjnie:

```
reservation
+
activation energy
```

FCR ma inną naturę aktywacji, zależną bezpośrednio od częstotliwości.

Dlatego kolejność:

```
DA
↓
aFRR
↓
mFRR
↓
ID
↓
rolling
↓
FCR
```

jest według mnie bezpieczniejsza niż próba zrobienia wszystkiego jednocześnie.

---

17. Zrób bardzo dużo testów constraintów

Po każdym solve automatycznie sprawdzaj:

$$SOC_{min} \leq SOC_t \leq SOC_{max}$$

$$P^{charge} + R^{DOWN} \leq P_{max}$$

$$P^{discharge} + R^{UP} \leq P_{max}$$

oraz:

```
no simultaneous charge/discharge
reserve commitments respected
gate closures respected
activation changes SOC
reservation does NOT change SOC
energy balance = 0
revenues add correctly
```

Na koniec każdego scenariusza zwracaj np.:

```
max_soc_violation
max_power_violation
max_energy_balance_violation
simultaneous_charge_discharge_hours
```

---

18. Zweryfikuj dane względem raportów PSE

PSE publikuje roczne statystyki wolumenów i średnich ważonych cen FCR/aFRR/mFRR.

Przykładowo raport za 2025 zawiera osobno średnie wolumeny i ceny MBP/MBU dla FCR, aFRR i mFRR w obu kierunkach.

Po pobraniu danych z API policz własne:

```
weighted_average_afrr_up
weighted_average_afrr_down
...
```

i sprawdź, czy wychodzą podobnie do PSE.

To jest świetny sanity check dla preprocessing.

Kolejność implementacji

Wersja	Co robisz	Rezultat
v0	zebranie i czyszczenie danych	master dataframe
v1	sam model fizyczny BESS	poprawny SoC
v2	DA arbitrage	pierwszy optimizer
v3	+ aFRR reservation	capacity stacking
v4	+ aFRR activation	SoC + activation revenue
v5	+ mFRR	pełniejsze balancing
v6	+ Intraday	DA + ID + balancing
v7	perfect foresight benchmark	maximum revenue
v8	rolling 1h / 24h	sequential optimizer
v9	+ FCR	pełny revenue stack
v10	sensitivities	wyniki pracy

Finalne eksperymenty do magisterki

Po zbudowaniu modelu zbadałbym przede wszystkim:

$$Value\ of\ Intraday$$

czyli DA + balancing vs DA + ID + balancing.

Dalej:

$$Value\ of\ Perfect\ Foresight$$

oraz wpływ częstotliwości reoptymalizacji:

```
rolling co 4h
rolling co 2h
rolling co 1h
rolling co 30 min
rolling co 15 min
```

Do tego sensitivity BESS:

```
1h battery
2h battery
4h battery
```

oraz różne:

```
degradation costs
efficiencies
SOC limits
initial SOC
```

Finalnie raportujesz:

```
total revenue
DA revenue
ID revenue
FCR capacity revenue
aFRR capacity revenue
aFRR activation revenue
mFRR capacity revenue
mFRR activation revenue

charging MWh
discharging MWh
activation UP MWh
activation DOWN MWh
equivalent full cycles
average SOC
reserve utilization
```

Struktura kodu

Od początku rozdzieliłbym projekt mniej więcej tak:

```
bess-thesis/
│
├── data/
│   ├── raw/
│   │   ├── pse/
│   │   └── tge/
│   └── processed/
│
├── src/
│   ├── data/
│   │   ├── pse.py
│   │   ├── tge.py
│   │   └── preprocess.py
│   │
│   ├── model/
│   │   ├── bess.py
│   │   ├── day_ahead.py
│   │   ├── intraday.py
│   │   ├── balancing.py
│   │   └── optimizer.py
│   │
│   ├── rolling/
│   │   └── rolling_optimizer.py
│   │
│   └── validation/
│       └── diagnostics.py
│
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_da_model.ipynb
│   ├── 03_balancing.ipynb
│   └── 04_results.ipynb
│
├── tests/
│
└── README.md
```

Notebooki traktujesz wtedy jako **eksperymenty i analizę**, a właściwa logika optymalizacji siedzi w `.py`.

Najbliższy konkretny krok

Na samym początku **nie pisz jeszcze optimizera**.

Najpierw przygotuj jeden dataframe dla np. **1–7 października 2025** zawierający:

```
datetime
DA price
```

```
aFRR capacity UP price
aFRR capacity DOWN price
aFRR procured UP MW
aFRR procured DOWN MW

mFRR capacity UP price
mFRR capacity DOWN price
mFRR procured UP MW
mFRR procured DOWN MW

aFRR activation UP MWh
aFRR activation DOWN MWh
aFRR activation UP price
aFRR activation DOWN price

mFRR activation UP MWh
mFRR activation DOWN MWh
mFRR activation UP price
mFRR activation DOWN price
```

RDB możesz na chwilę zostawić puste.

Jeżeli ten tygodniowy dataset będzie poprawny, następny krok to już bardzo prosty **DA-only BESS optimizer**. Potem dokładamy rynek bilansujący warstwa po warstwie.